МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №2
по дисциплине «Построение и анализ алгоритмов»
Тема: Коммивояжер.

Студент гр. 4343 _____ Маркашов Г.Д.

Преподаватель _____ Жангиров Т. Р.

Санкт-Петербург
2026

Цель Работы.

Изучить методы решения задачи коммивояжёра на полном взвешенном графе. Реализовать на любом доступном языке программирования точный метод ветвей и границ (алгоритм Литтла) и приближённый 2 – приближённый алгоритм на основе минимального остовного дерева для нахождения маршрута коммивояжёра и его стоимости по заданной матрице весов графа. Провести тестирование реализованных алгоритмов на различных входных данных и проверить корректность полученных результатов.

Задание.

Вариант 1.

МВиГ: Алгоритм Литтла. Приближённый алгоритм: 2-приближение по МОД (АДО МОД).

Замечание к варианту 1. АДО МОД является 2-приближением только для евклидовой матрицы. Начинать обход МОД со стартовой вершины.

Коммивояжёр: 2-приближённый алгоритм.

Разработайте программу, которая решает задачу коммивояжера при помощи 2-приближенного алгоритма. В данной постановке задачи нужно вернуться в исходную вершину после прохождения всех остальных вершин. При обходе остовного дерева (MST) необходимо идти по минимальному допустимому ребру из текущего. Каждая вершина в графе обозначается неотрицательным числом, начиная с 0, каждое ребро имеет неотрицательный вес. В графе нет рёбер из вершины в саму себя, в матрице весов на месте таких отсутствующих рёбер стоит значение -1.

Коммивояжёр: ветви и границы

В волшебной стране Алгоритмии великий маг, Гамильтон, задумал невероятное путешествие, чтобы связать все города страны заклятием процветания. Для этого ему необходимо посетить каждый город ровно один раз, создавая тропу благополучия, и вернуться обратно в столицу, используя минимум своих чародейских сил. Вашей задачей является помощь в

прокладывании маршрута с помощью древнего и могущественного алгоритма ветвей и границ.

Карта дорог Алгоритмии перед Гамильтоном представляет собой полный граф, где каждый город соединён магическими порталами с каждым другим. Стоимость использования портала из города в город занимает определённое количество маны, и Гамильтон стремится минимизировать общее потребление магической энергии для закрепления проклятия.

Теоретический минимум

Задача коммивояжёра (TSP, Travelling Salesman Problem) — задача нахождения кратчайшего гамильтонова цикла в полном взвешенном графе. Необходимо посетить все вершины ровно один раз, начиная и заканчивая маршрут в одной и той же вершине, минимизируя суммарный вес пройденного пути.

Граф задаётся матрицей весов, где каждый элемент $M_{i,j}$ определяет стоимость перехода из вершины i в вершину j.

В данной лабораторной работе задача решается двумя методами: точным методом ветвей и границ (алгоритм Литтла) и приближённым 2-приближённым алгоритмом на основе минимального остовного дерева (МОД).

1. Метод ветвей и границ. Алгоритм Литтла

Метод ветвей и границ позволяет находить оптимальное решение задачи коммивояжёра за счёт перебора возможных маршрутов с отсечением заведомо невыгодных вариантов.

Основная идея работы алгоритма:

1) Исходная матрица стоимости редуцируется:
    - из каждой строки вычитается минимальный элемент;
    - затем из каждого столбца вычитается минимальный элемент.

2) Сумма вычтенных значений образует нижнюю границу стоимости маршрута.
3) На каждом шаге выбирается нуль с максимальным штрафом, который соответствует наиболее критическому ребру для выбора в маршруте.
4) Задача разбивается на две подзадачи:
   - левая ветвь с включением ребра;
   - правая ветвь с его исключением.
5) Для каждой ветви снова вычисляется нижняя граница.
6) Если нижняя граница ветви больше уже найденного решения, ветвь отсекается.
7) Процесс продолжается до построения гамильтонова цикла.
8) Итоговый маршрут восстанавливается из выбранных рёбер после завершения рекурсивного перебора.

2. Приближённый алгоритм. 2-приближение на основе МОД

Метод позволяет получить решение быстрее, но не обязательно оптимальное.

Основная идея работы алгоритма:

1) По исходному графу строится минимальное остовное дерево (MST), например, алгоритмом Прима.
2) Выполняется обход дерева в порядке предварительного DFS (preorder), начиная со стартовой вершины.
3) Маршрут формируется из посещённых вершин с возвратом в исходную вершину.
4) Данный алгоритм является 2-приближением для метрики Евклида: $\text{длина маршрута} \leq 2 \times \text{оптимальная}$.

Временная сложность:

• Алгоритм Литтла: в худшем случае алгоритм проверяет все возможные перестановки рёбер, что соответствует $O(n!)$, где n – число вершин графа. Практически время работы уменьшается за счёт отсечений и

выбора критических рёбер. С увеличением числа вершин время растёт экспоненциально.

- 2-приближение по MST: построение MST – $O(n^2)$, сортировка соседей на каждом шаге *DFS* – $O(n\log n)$ в худшем случае для всех вершин и обход дерева – $O(n)$. Общая сложность $O(n^2)$, что делает алгоритм применимым для больших графов (сотни и тысячи вершин).

Пространственная сложность:

- Алгоритм Литтла: хранение матриц редукции на каждом шаге рекурсии – $O(n^2)$для каждой копии, стек рекурсивных вызовов – глубина до $O(n)$, хранение текущего списка выбранных рёбер и лучшего найденного маршрута – $O(n)$ и $O(n)$ соответственно. Итоговая пространственная сложность примерно $O(n^2)$.

- 2-приближение по MST: хранение матрицы весов – $O(n^2)$, хранение MST в виде списка смежности – $O(n)$для дерева с $n-1$ рёбрами, стек рекурсии для DFS – глубина до $O(n)$. Итоговая пространственная сложность: $O(n^2)$.

Выполнение работы.

1. Приближённый алгоритм. 2-приближение на основе МОД

Программа на языке программирования Python реализовывает 2 приближенный алгоритм решения коммивояжера. Алгоритм гарантирует, что длина найденного пути не превышает удвоенную длину оптимального пути.

INF – глобальная переменная с значение бесконечности

read_data() – функция считывания данных с консоли.

create_mst(matrix: list[list[float]], start_vertex_idx: int) -> tuple[list[int], list[float]] – функция постройки MST дерева.

build_adjacency(parent: list[int], parent_weight: list[float]) -> list[list[tuple[int, float]]] – функция постройки матрицы смежности

dfs_preorder(adj: list[list[tuple[int, float]]], start: int) -> list[int] – функция обхода в глубину

path_len(path: list[int], matrix: list[list[float]]) – функция подсчета длины пути

2. Алгоритм Литтла

Программа реализует метод ветвей и границ для точного решения задачи коммивояжера. Алгоритм гарантирует оптимальное нахождение пути гамильтонова цикла с минимальной стоимостью.

INF – глобальная переменная с значение бесконечности.

read_data(n: int) – функция по считыванию данных с консоли.

matrix_copy(*matrix*: *list*[list[*float*]]) -> *list*[list[*float*]] – функция копирования матрицы

reduce_matrix(matrix: list[list[float]]) -> float – функция редуцирования матрицы и возврата нижней границы по полученной матрице. Редуцирование матрицы означает последовательное вычитание минимального элемента из каждой строки и столбца, за исключение значений равных бесконечности.

remove_row_col(*matrix*: *list*[list[*float*]], *rows*: *list*[*int*], *cols*: *list*[*int*], *row_idx*: *int*, *col_idx*: *int*) -> *tuple*[list[list[*float*]], list[*int*], list[*int*]] - функция удаления столбца и колонки из матрицы, необходимо для ветвления и достижения матрицы 1x1.

forbid_edge(*matrix*: *list*[list[*float*]], *rows*: *list*[*int*], *cols*: *list*[*int*], *from_city*: *int*, *to_city*: *int*) – функция запрета обратного ребра для предотвращения ранних циклов.

estimate_zero(*matrix*: *list*[list[*float*]], *n*: *int*, *i*: *int*, *j*: *int*) – функция оценивания нуля

find_best_zero(*matrix*: *list*[list[*float*]]) – функция поиска нуля с максимальной оценкой

reconstruct_path(*edges*: *list*[tuple[*int*, *int*]], *n*: *int*) -> *list*[*int*] – функция реконструирования пути

find_chain_endpoints(*edges*: *list*[tuple[*int*, *int*]], *u*: *int*, *v*: *int*) - функция по постройки цепи пути.

tsp_branch_and_bound(*matrix*: *list*[list[*float*]]) – основная функция по реализации метода ветвей и границ.

rec_branch_and_bound(*matrix*: *list*[list[*float*]], *rows*: *list*[*int*], *cols*: *list*[*int*], *edges*: *list*[tuple[*int*, *int*]], *bound*: *float*) – рекурсивная функция по распределению на ветви и подчета границы.

Код выполнения работы см. в Приложении А.

Приложения

Приложение А

littl.py:

```
INF = float("+inf")

def read_data(n: int) -> list[list[float]]:
    matrix = []
    for _ in range(n):
        line = [INF if x == "-1" else float(x) for x in
input().split(" ")]
        matrix.append(line)
    return matrix

def matrix_copy(matrix: list[list[float]]) -> list[list[float]]:
    return [row[:] for row in matrix]

def reduce_matrix(matrix: list[list[float]]) -> float:
    lower_bound = 0
    n = len(matrix)

    # вычитаем миниму из строк
    for i in range(n):
        min_val = min(matrix[i])
        if min_val == 0:
            continue
        if min_val != INF:
            lower_bound += min_val
            for j in range(n):
                if matrix[i][j] != INF:
                    matrix[i][j] -= min_val

    # вычитаем минимум из столбцов
    for j in range(n):
        min_val = min(matrix[i][j] for i in range(n))
        if min_val == 0:
            continue
        if min_val != INF:
            lower_bound += min_val
            for i in range(n):
                if matrix[i][j] != INF:
                    matrix[i][j] -= min_val

    return lower_bound

def remove_row_col(matrix: list[list[float]], rows: list[int], cols:
list[int],
```

```
                    row_idx: int, col_idx: int) ->
tuple[list[list[float]], list[int], list[int]]:
    n = len(matrix)
    new_matrix = []
    new_rows = []
    new_cols = []

    for i in range(n):
        new_row = []
        if i != row_idx:
            for j in range(n):
                if j != col_idx:
                    new_row.append(matrix[i][j])
            new_matrix.append(new_row)
            new_rows.append(rows[i])

    for j in range(n):
        if j != col_idx:
            new_cols.append(cols[j])

    return new_matrix, new_rows, new_cols

def forbid_edge(matrix: list[list[float]], rows: list[int], cols:
list[int],
                from_city: int, to_city: int) -> None:
    n = len(matrix)
    row_idx = col_idx = -1

    for i in range(n):
        if rows[i] == from_city:
            row_idx = i
            break

    for j in range(n):
        if cols[j] == to_city:
            col_idx = j
            break

    if row_idx != -1 and col_idx != -1:
        matrix[row_idx][col_idx] = INF

def estimate_zero(matrix: list[list[float]], n: int, i: int, j: int) -
> float:
    row_min = INF
    for k in range(n):
        if k != j and matrix[i][k] != INF:
            row_min = min(row_min, matrix[i][k])
    if row_min == INF:
        row_min = 0
```

```
    col_min = INF
    for k in range(n):
        if k != i and matrix[k][j] != INF:
            col_min = min(col_min, matrix[k][j])
    if col_min == INF:
        col_min = 0

    return row_min + col_min

def find_best_zero(matrix: list[list[float]]) -> tuple[tuple[int, int]
| None, float]:
    n = len(matrix)
    best_pos = None
    best_penalty = -1

    for i in range(n):
        for j in range(n):
            if matrix[i][j] != 0:
                continue
            penalty = estimate_zero(matrix, n, i, j)
            if penalty > best_penalty:
                best_penalty = penalty
                best_pos = (i, j)

    return best_pos, best_penalty

def reconstruct_path(edges: list[tuple[int, int]], n: int) ->
list[int]:
    next_map = {u: v for u, v in edges}

    path = [0]
    cur = 0

    for _ in range(n - 1):
        cur = next_map[cur]
        path.append(cur)

    return path

def find_chain_endpoints(edges: list[tuple[int, int]], u: int, v: int)
-> tuple[int, int]:
    next_map = {}
    prev_map = {}

    for a, b in edges:
        next_map[a] = b
        prev_map[b] = a

    next_map[u] = v
    prev_map[v] = u
```

```
    # Ищем начало
    start = u
    while start in prev_map:
        start = prev_map[start]

    # Ищем конец
    end = v
    while end in next_map:
        end = next_map[end]

    return start, end


def creates_small_cycle(edges: list[tuple[int, int]], u: int, v: int,
total_n: int) -> bool:
    next_map = {a: b for a, b in edges}
    next_map[u] = v

    cur = v
    length = 1

    while cur in next_map:
        cur = next_map[cur]
        length += 1
        if cur == u:
            return length < total_n

    return False

def tsp_branch_and_bound(matrix: list[list[float]]):
    n = len(matrix)
    orig = matrix_copy(matrix)

    reduced = matrix_copy(matrix)
    start_bound = reduce_matrix(reduced)

    rows = list(range(n))
    cols = list(range(n))

    best_cost = INF
    best_path = None

    def rec_branch_and_bound(matrix: list[list[float]], rows:
list[int], cols: list[int],
                  edges: list[tuple[int, int]], bound: float):
        nonlocal best_cost, best_path

        if bound >= best_cost:
            return
```

```
        if len(matrix) == 1:
            u = rows[0]
            v = cols[0]
            final_edges = edges + [(u, v)]
            path = reconstruct_path(final_edges, n)
            cost = sum(orig[path[i]][path[(i + 1) % n]] for i in
range(n))

            if cost < best_cost:
                best_cost = cost
                best_path = path
            return

        pos, penalty = find_best_zero(matrix)
        if pos is None:
            return

        i, j = pos
        u = rows[i]
        v = cols[j]

        force_left = penalty >= INF / 2

        # ветка 1 - не включаем ребро (u, v)
        if not force_left:
            right_matrix = matrix_copy(matrix)
            right_matrix[i][j] = INF
            right_bound = bound + reduce_matrix(right_matrix)

            rec_branch_and_bound(right_matrix, rows, cols, edges,
right_bound)

        # ветка 2 - включаем ребро (u, v)
        if not creates_small_cycle(edges, u, v, n):
            left_matrix = matrix_copy(matrix)
            start, end = find_chain_endpoints(edges, u, v)
            left_matrix, left_rows, left_cols =
remove_row_col(left_matrix, rows, cols, i, j)
            forbid_edge(left_matrix, left_rows, left_cols, end, start)
            left_bound = bound + reduce_matrix(left_matrix)

            rec_branch_and_bound(left_matrix, left_rows, left_cols,
edges + [(u, v)], left_bound)

    rec_branch_and_bound(reduced, rows, cols, [], start_bound)

    return best_path, best_cost

if __name__ == "__main__":
```

```
    n = int(input())
    matrix = read_data(n)

    path, cost = tsp_branch_and_bound(matrix)
    print(' '.join(map(str, path)))
    print(cost)
```

tsp_2approx.py:

```
import sys

INF = float("+inf")

def read_data() -> list[list[float]]:
    lines = sys.stdin.read().strip().split('\n')
    start_vertex = int(lines[0])

    matrix = []
    for i in range(1, len(lines)):
        row = []
        for value in map(float, lines[i].split()):
            if value < 0:
                row.append(INF)
            else:
                row.append(value)
        matrix.append(row)

    return start_vertex, matrix

def create_mst(matrix: list[list[float]], start_vertex_idx: int) ->
tuple[list[int], list[float]]:
    n = len(matrix)
    vistited = []
    min_edge = [INF] * n
    parent = [-1] * n
    parent_weight = [0] * n

    min_edge[start_vertex_idx] = 0

    for _ in range(n):
        u = -1
        best = INF

        for i in range(n):
            if i not in vistited and min_edge[i] < best:
                best = min_edge[i]
                u = i

        if u == -1:
            break

        vistited.append(u)

        for v in range(n):
            if v not in vistited and matrix[u][v] < min_edge[v]:
                min_edge[v] = matrix[u][v]
                parent[v] = u
```

```
                parent_weight[v] = matrix[u][v]

    return parent, parent_weight


def build_adjacency(parent: list[int], parent_weight: list[float]) ->
list[list[tuple[int, float]]]:
    n = len(parent)
    adj = [[] for _ in range(n)]
    for v in range(n):
        p = parent[v]
        if p != -1:
            w = parent_weight[v]
            adj[p].append((v, w))
            adj[v].append((p, w))
    return adj

def dfs_preorder(adj: list[list[tuple[int, float]]], start: int) ->
list[int]:
    visited = []
    path = []

    def dfs(v: int):
        visited.append(v)
        path.append(v)

        for neighbor, weight in sorted(adj[v], key=lambda x: (x[1],
x[0])):
            if neighbor not in visited:
                dfs(neighbor)

    dfs(start)
    return path

def path_len(path: list[int], matrix: list[list[float]]):
    total = 0
    for i in range(len(path) - 1):
        total += matrix[path[i]][path[i+1]]
    return total

if __name__ == "__main__":
    start_vertex_idx, matrix = read_data()

    parent, parent_weight = create_mst(matrix, start_vertex_idx)
    adj = build_adjacency(parent, parent_weight)
    path = dfs_preorder(adj, start_vertex_idx)
    path.append(start_vertex_idx)

    l = path_len(path, matrix)

    print(f"{l:.2f}")
    print(" ".join(list(map(str, path))))
```